FACULDADE DE TECNOLOGIA DE INDAIATUBA DR. ARCHIMEDES LAMMOGLIA

MAIARA MACEDO RIBEIRO DE SOUSA

TUTORIAL: LOCALIZAÇÃO (GPS) NO ANDROID COM JAVA

INDAIATUBA

2026

SUMÁRIO

Sumário

INTRODUÇÃO **3**

1. FUNDAMENTAÇÃO TEÓRICA **3**

1.1 Provedor de localização combinada 3

1.2 Permissões 3

1.3 Configuração do pedido 4

1.4 Classes utilizadas 4

1.5 Fluxo do aplicativo 4

1.6 Rotas 5

1.7 Boas práticas aplicadas 5

2. PASSO A PASSO DA IMPLEMENTAÇÃO **6**

2.1 Criação do projeto 6

2.2 Adição da biblioteca de localização 7

2.3 Declaração das permissões 8

2.4 Criação da tela 9

2.5 Escrita da lógica 11

2.6 Preparação do celular 14

2.7 Execução do aplicativo 15

3. TESTES E RESULTADOS **16**

4. CONSIDERAÇÕES FINAIS **19**

5. REFERÊNCIAS **20**

INTRODUÇÃO

Este tutorial apresenta o recurso de localização (GPS) dos smartphones Android e o passo a passo para utilizá-lo em um aplicativo. O trabalho foi realizado na disciplina Programação para Dispositivos Móveis I, cuja atividade pede a escolha de um recurso do aparelho e a criação de um aplicativo que o implemente.

O aplicativo desenvolvido mostra em tempo real a latitude, a longitude e a precisão da posição do aparelho. A partir dessa posição, ele abre no Google Maps a rota até um destino digitado pelo usuário.

O projeto utiliza Java e layout XML no Android Studio, com a API de localização do Google Play Services.

1. FUNDAMENTAÇÃO TEÓRICA

No Android, o aplicativo não se comunica diretamente com o GPS. Ele solicita localizações a um serviço do sistema, depois de receber a permissão do usuário.

1.1 Provedor de localização combinada

O `FusedLocationProviderClient`, do Google Play Services, é a API utilizada neste aplicativo. O desenvolvedor informa a qualidade desejada, e a API gerencia as tecnologias envolvidas, como GPS e Wi-Fi.

1.2 Permissões

A localização é um dado sensível. Por isso, o uso desse recurso exige declaração no manifesto e autorização do usuário durante a execução. O Quadro 1 apresenta os dois níveis de precisão definidos pela documentação (ANDROID DEVELOPERS).

Quadro 1 – Níveis de permissão de localização

Nível	**Permissão**	**Precisão informada**
Aproximada	`ACCESS_COARSE_LOCATION`	Área de cerca de 3 km²
Precisa	`ACCESS_FINE_LOCATION`	Em geral, cerca de 50 m; às vezes, poucos metros ou menos

Fonte: elaborado pela autora (2026).

Este aplicativo usa a localização em primeiro plano, isto é, com a tela visível. Assim, não precisa da permissão `ACCESS_BACKGROUND_LOCATION`.

1.3 Configuração do pedido

O `LocationRequest` define o intervalo entre as atualizações e a prioridade, que indica quais fontes de localização usar (ANDROID DEVELOPERS). O Quadro 2 resume as prioridades disponíveis.

Quadro 2 – Prioridades de localização

Prioridade	**Precisão**	**Observação**
`PRIORITY_HIGH_ACCURACY`	A mais alta possível	Maior chance de usar GPS
`PRIORITY_BALANCED_POWER_ACCURACY`	Cerca de 100 m	Tende a usar Wi-Fi e torres de celular
`PRIORITY_LOW_POWER`	Cerca de 10 km	Baixo consumo de energia
`PRIORITY_PASSIVE`	Não se aplica	Recebe apenas localizações pedidas por outros aplicativos

Fonte: elaborado pela autora (2026).

Este aplicativo usa `PRIORITY_HIGH_ACCURACY`, com intervalo de 5 segundos. O intervalo é uma preferência: as atualizações podem chegar antes, depois ou nem chegar, caso não haja conectividade.

1.4 Classes utilizadas

O Quadro 3 apresenta as principais classes usadas no aplicativo.

Quadro 3 – Classes utilizadas no aplicativo

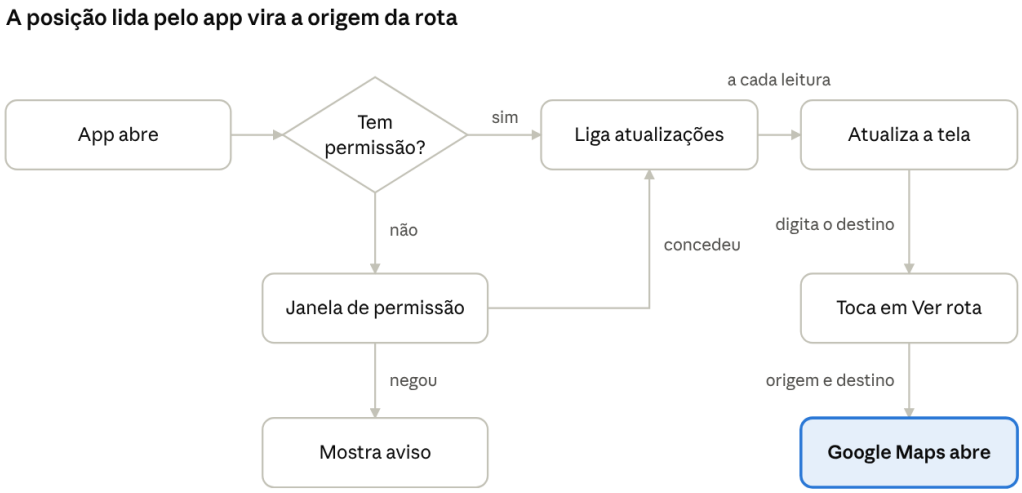
Classe	**Função no aplicativo**
`FusedLocationProviderClient`	Inicia e encerra as atualizações de localização
`LocationRequest`	Guarda o intervalo e a prioridade do pedido
`LocationCallback`	Recebe as novas localizações em `onLocationResult()`
`Location`	Traz a latitude, a longitude e a precisão
`Intent`	Abre o link da rota no Google Maps

Fonte: elaborado pela autora (2026).

1.5 Fluxo do aplicativo

O aplicativo busca a localização ao abrir, depois de confirmar a permissão. A posição recebida fica guardada para servir de origem da rota, como mostra a Figura 1.

Figura 1 – Fluxo de funcionamento do aplicativo



Fonte: elaborado pela autora (2026).

Ao sair da tela, o aplicativo remove as atualizações, para economizar bateria. Ao voltar, ele as liga novamente (ANDROID DEVELOPERS).

1.6 Rotas

O cálculo da rota é feito pelo Google Maps, e o aplicativo envia a origem e o destino por um link. Esse formato, chamado Maps URLs, não exige chave de API. No Android, o link abre o aplicativo Google Maps quando ele está instalado; caso contrário, abre o navegador (GOOGLE FOR DEVELOPERS). O formato utilizado é o seguinte:

```
https://www.google.com/maps/dir/?api=1&origin=LATITUDE,LONGITUDE&destination=DESTINO
```

O destino passa pelo método `Uri.encode()`, pois os valores de uma URL precisam ser codificados. O aplicativo abre o link com um `Intent` do tipo `ACTION_VIEW`.

1.7 Boas práticas aplicadas

- Pedir as permissões ao abrir o aplicativo, pois a localização é a função principal da tela.
- Remover as atualizações em `onPause()` e retomá-las em `onResume()`.
- Tratar a negativa da permissão sem encerrar o aplicativo.

- Funcionar também com a localização aproximada.

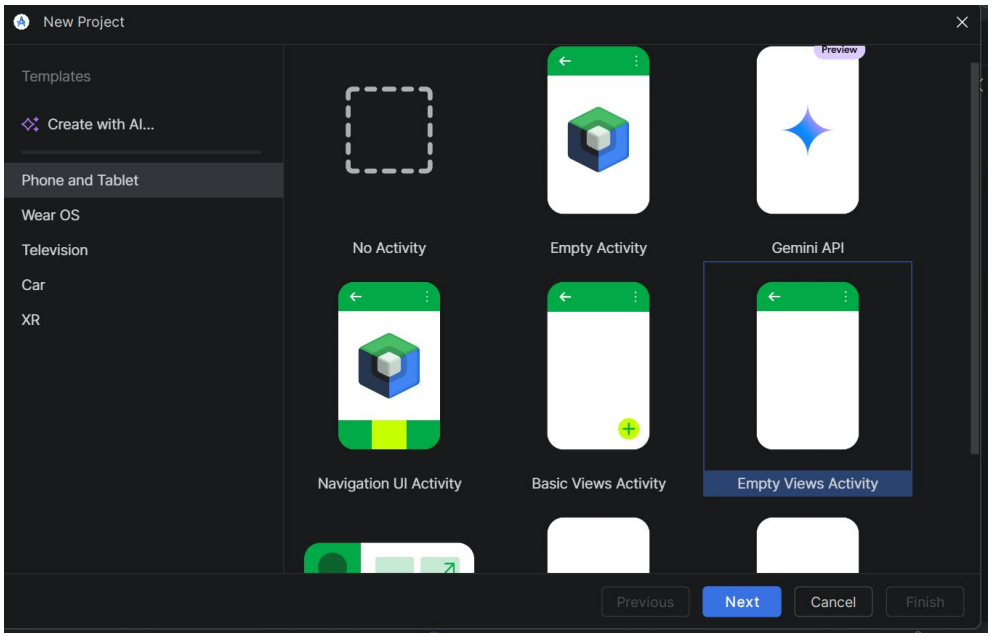
2. PASSO A PASSO DA IMPLEMENTAÇÃO

Esta seção descreve, em sete etapas, a construção do aplicativo, do projeto vazio à execução no celular.

2.1 Criação do projeto

No Android Studio, foi aberto o menu **File > New > New Project**. Em **Phone and Tablet**, foi escolhido o modelo **Empty Views Activity** (Figura 2).

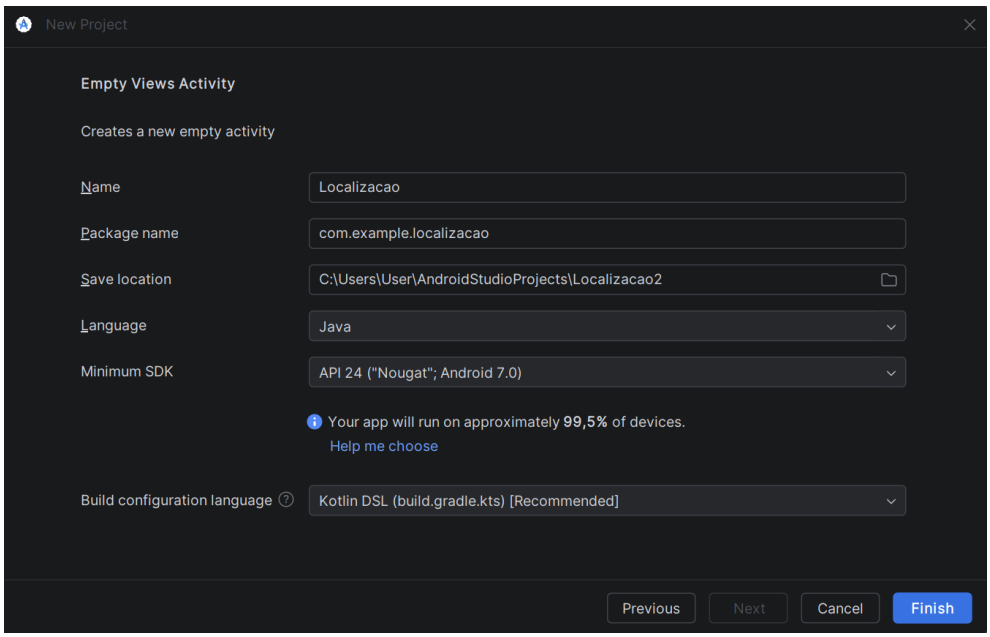
Figura 2 – Escolha do modelo Empty Views Activity



Fonte: elaborado pela autora (2026).

Em seguida, o projeto foi configurado com o nome Localizacao, o pacote `com.example.localizacao`, a linguagem Java e o Minimum SDK API 24 (Figura 3).

Figura 3 – Configuração do projeto



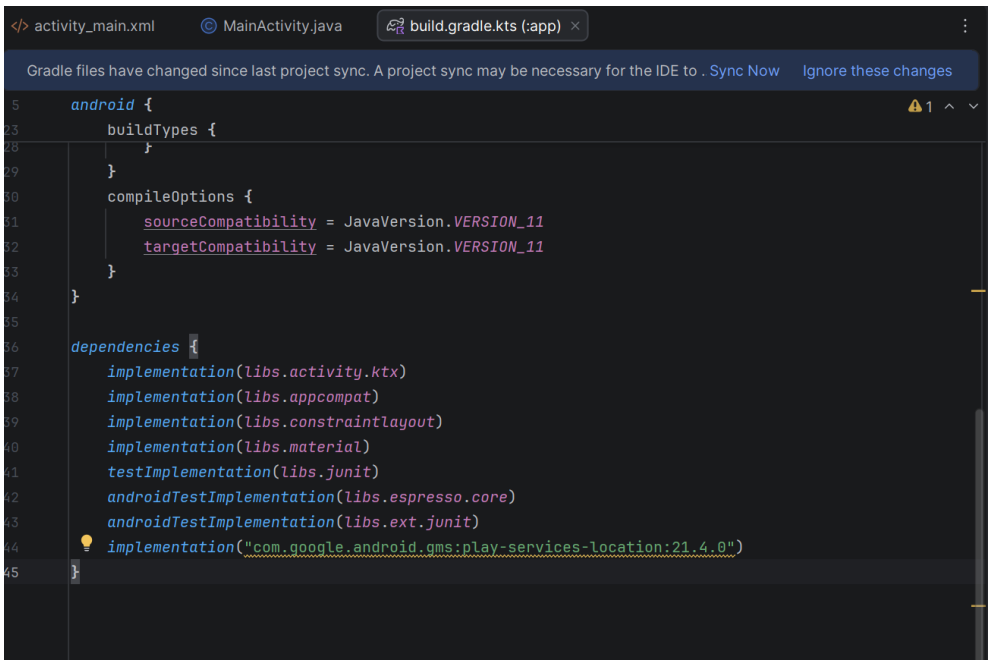
Fonte: elaborado pela autora (2026).

2.2 Adição da biblioteca de localização

A API de localização faz parte do Google Play Services e precisa ser adicionada ao projeto (GOOGLE FOR DEVELOPERS). Para isso, a dependência `play-services-location` foi incluída no arquivo `build.gradle.kts (Module :app)` (Figura 4). Depois, a opção **Sync Now** baixou a biblioteca para o projeto. A linha adicionada foi:

```
implementation("com.google.android.gms:play-services-location:21.4.0")
```

Figura 4 – Dependência adicionada ao build.gradle.kts



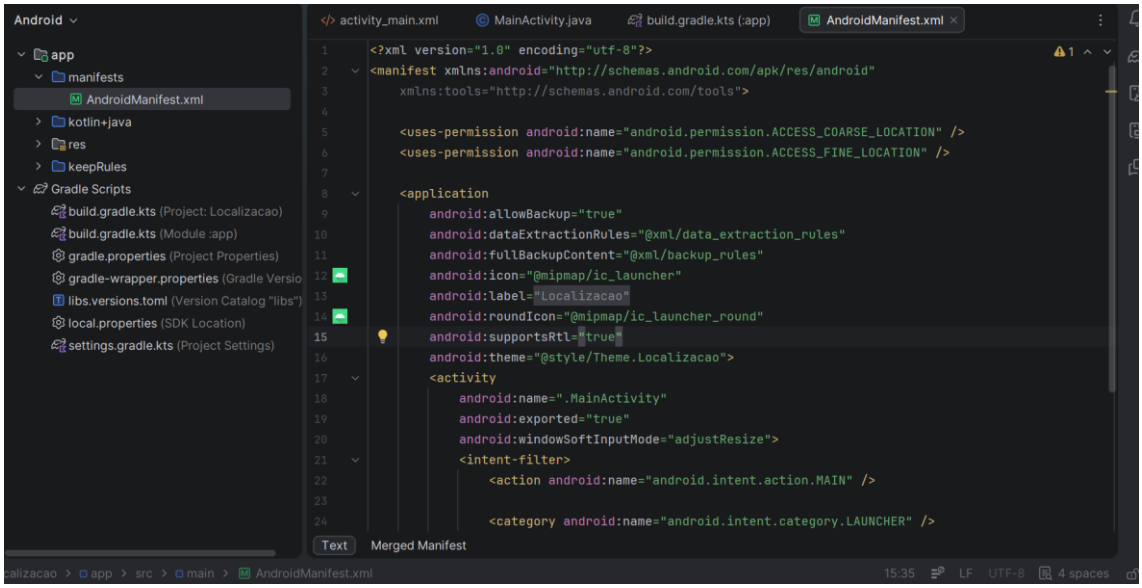
Fonte: elaborado pela autora (2026).

2.3 Declaração das permissões

No arquivo `AndroidManifest.xml`, as duas permissões de localização foram declaradas antes da tag `<application>` (Figura 5):

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

Figura 5 – Permissões declaradas no AndroidManifest.xml

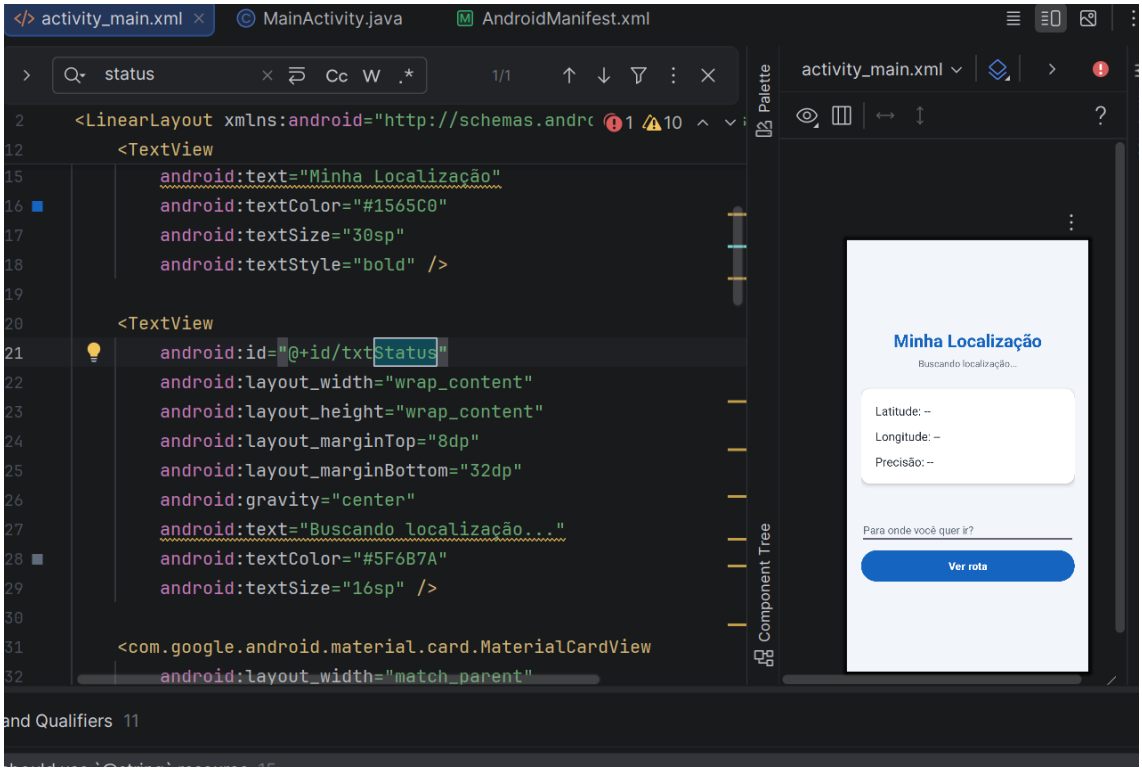


Fonte: elaborado pela autora (2026).

2.4 Criação da tela

A tela foi criada no arquivo `activity_main.xml`, com um título, um texto de status, um cartão com os dados da localização, um campo para o destino e o botão Ver rota (Figura 6). Cada item recebeu um `android:id`, que permite ao Java alterar o seu conteúdo.

Figura 6 – Layout da tela no modo Split



Fonte: elaborado pela autora (2026).

O código completo do arquivo `activity_main.xml` é o seguinte:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#F2F5F9"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Minha Localização"
        android:textColor="#1565C0"
        android:textSize="30sp"
        android:textStyle="bold" />
```

```
    <TextView
        android:id="@+id/txtStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:layout_marginBottom="32dp"
        android:gravity="center"
        android:text="Buscando localização..."
        android:textColor="#5F6B7A"
        android:textSize="16sp" />

    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="32dp"
        app:cardBackgroundColor="#FFFFFF"
        app:cardCornerRadius="16dp"
        app:cardElevation="4dp"
        app:strokeWidth="0dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="24dp">

            <TextView
                android:id="@+id/txtLatitude"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Latitude: --"
                android:textColor="#1F2933"
                android:textSize="20sp" />

            <TextView
                android:id="@+id/txtLongitude"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="16dp"
                android:text="Longitude: --"
                android:textColor="#1F2933"
                android:textSize="20sp" />

            <TextView
                android:id="@+id/txtPrecisao"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="16dp"
                android:text="Precisão: --"
                android:textColor="#1F2933"
                android:textSize="20sp" />

        </LinearLayout>

    </com.google.android.material.card.MaterialCardView>

    <EditText
        android:id="@+id/edtDestino"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:hint="Para onde você quer ir?"
        android:inputType="text"
```

```
        android:textColor="#1F2933"
        android:textColorHint="#5F6B7A" />

    <Button
        android:id="@+id/btnRota"
        android:layout_width="match_parent"
        android:layout_height="60dp"
        android:layout_marginTop="8dp"
        android:backgroundTint="#1565C0"
        android:text="Ver rota"
        android:textColor="#FFFFFF"
        android:textSize="18sp"
        android:textStyle="bold" />

</LinearLayout>
```

Para a hora e os ícones do topo ficarem visíveis sobre o fundo claro, a linha abaixo foi adicionada aos arquivos `themes.xml` e `themes.xml (night)`:

```
<item name="android:windowLightStatusBar">true</item>
```

2.5 Escrita da lógica

A lógica do aplicativo foi escrita no arquivo `MainActivity.java`. O início do arquivo traz as importações, as variáveis e o tratamento da resposta da permissão:

```
package com.example.localizacao;

import android.Manifest;
import android.annotation.SuppressLint;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.location.Location;
import android.net.Uri;
import android.os.Bundle;
import android.os.Looper;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;

import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationCallback;
import com.google.android.gms.location.LocationRequest;
import com.google.android.gms.location.LocationResult;
import com.google.android.gms.location.LocationServices;
import com.google.android.gms.location.Priority;

import java.util.Locale;

public class MainActivity extends AppCompatActivity {

    private FusedLocationProviderClient fusedLocationClient;
    private LocationRequest locationRequest;
```

```
    private LocationCallback locationCallback;

    private TextView txtStatus;
    private TextView txtLatitude;
    private TextView txtLongitude;
    private TextView txtPrecisao;
    private EditText edtDestino;

    private boolean rastreando = false;
    private Location ultimaLocalizacao;

    // Mostra a janela de permissao e recebe a resposta do usuario.
    private final ActivityResultLauncher<String[]> pedirPermissao =
            registerForActivityResult(new
ActivityResultContracts.RequestMultiplePermissions(), resultado -> {
                if (temPermissao()) {
                    iniciar();
                } else {
                    txtStatus.setText("Permissão negada");
                }
            });
```

O método `onCreate()` liga a tela ao código, cria o cliente e o pedido de localização, define o callback e inicia a busca:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        txtStatus = findViewById(R.id.txtStatus);
        txtLatitude = findViewById(R.id.txtLatitude);
        txtLongitude = findViewById(R.id.txtLongitude);
        txtPrecisao = findViewById(R.id.txtPrecisao);
        edtDestino = findViewById(R.id.edtDestino);
        Button btnRota = findViewById(R.id.btnRota);
        btnRota.setOnClickListener(v -> verRota());

        fusedLocationClient =
LocationServices.getFusedLocationProviderClient(this);

        // Pede uma atualizacao a cada 5 segundos, com a maior precisao
possivel.
        locationRequest = new
LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY, 5000).build();

        // O sistema chama este metodo sempre que ha uma nova localizacao.
        locationCallback = new LocationCallback() {
            @Override
            public void onLocationResult(@NonNull LocationResult locationResult)
{
                Location location = locationResult.getLastLocation();
                if (location != null) {
                    ultimaLocalizacao = location;
                    txtLatitude.setText(String.format(Locale.getDefault(),
"Latitude: %.6f", location.getLatitude()));
                    txtLongitude.setText(String.format(Locale.getDefault(),
"Longitude: %.6f", location.getLongitude()));
                    txtPrecisao.setText(String.format(Locale.getDefault(),
"Precisão: %.0f m", location.getAccuracy()));
                    txtStatus.setText("Localização encontrada. Digite o
destino.");
                }
```

```
            }
        };

        // Busca a localizacao assim que o app abre.
        if (temPermissao()) {
            iniciar();
        } else {
            pedirPermissao.launch(new String[] {
                    Manifest.permission.ACCESS_FINE_LOCATION,
                    Manifest.permission.ACCESS_COARSE_LOCATION
            });
        }
    }
```

Em seguida, ficam os métodos ligados ao ciclo de vida e à permissão. As atualizações são iniciadas por `requestLocationUpdates()` (ANDROID DEVELOPERS):

```
    @Override
    protected void onResume() {
        super.onResume();
        if (rastreando && temPermissao()) {
            comecarAtualizacoes();
        }
    }

    @Override
    protected void onPause() {
        super.onPause();
        // Para de receber localizacoes para economizar bateria.
        fusedLocationClient.removeLocationUpdates(locationCallback);
    }

    private boolean temPermissao() {
        return ContextCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_FINE_LOCATION)
                == PackageManager.PERMISSION_GRANTED
                || ContextCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_COARSE_LOCATION)
                == PackageManager.PERMISSION_GRANTED;
    }

    private void iniciar() {
        rastreando = true;
        txtStatus.setText("Buscando localização...");
        comecarAtualizacoes();
    }

    // A permissao ja foi conferida antes de este metodo ser chamado.
    @SuppressLint("MissingPermission")
    private void comecarAtualizacoes() {
        fusedLocationClient.requestLocationUpdates(locationRequest,
locationCallback, Looper.getMainLooper());
    }
```

Por fim, o método `verRota()` valida os dados, monta o link com a origem e o destino e abre o Google Maps:

```
    // Abre o Google Maps com a rota da posicao atual ate o destino digitado.
    private void verRota() {
```

```
        String destino = edtDestino.getText().toString().trim();

        if (ultimaLocalizacao == null) {
            Toast.makeText(this, "Aguarde a localização",
Toast.LENGTH_SHORT).show();
            return;
        }
        if (destino.isEmpty()) {
            Toast.makeText(this, "Digite um destino",
Toast.LENGTH_SHORT).show();
            return;
        }

        String endereco = "https://www.google.com/maps/dir/?api=1&origin="
                + ultimaLocalizacao.getLatitude() + "," +
ultimaLocalizacao.getLongitude()
                + "&destination=" + Uri.encode(destino);

        startActivity(new Intent(Intent.ACTION_VIEW, Uri.parse(endereco)));
    }
}
```

O Quadro 4 resume a função de cada parte do código.

Quadro 4 – Partes do código

Parte	**Função**
`pedirPermissao`	Abre a janela de permissão e trata a resposta do usuário
`onCreate()`	Liga a tela ao código, cria o cliente e o pedido, e inicia a busca
`onLocationResult()`	Guarda a posição e mostra os dados na tela
`onResume()` e `onPause()`	Retomam e removem as atualizações conforme a tela aparece ou sai
`temPermissao()`	Confere se o aplicativo tem alguma das duas permissões
`iniciar()`	Marca o rastreamento como ativo e atualiza o status
`comecarAtualizacoes()`	Chama `requestLocationUpdates()`
`verRota()`	Valida os dados, monta o link e abre o Google Maps

Fonte: elaborado pela autora (2026).

2.6 Preparação do celular

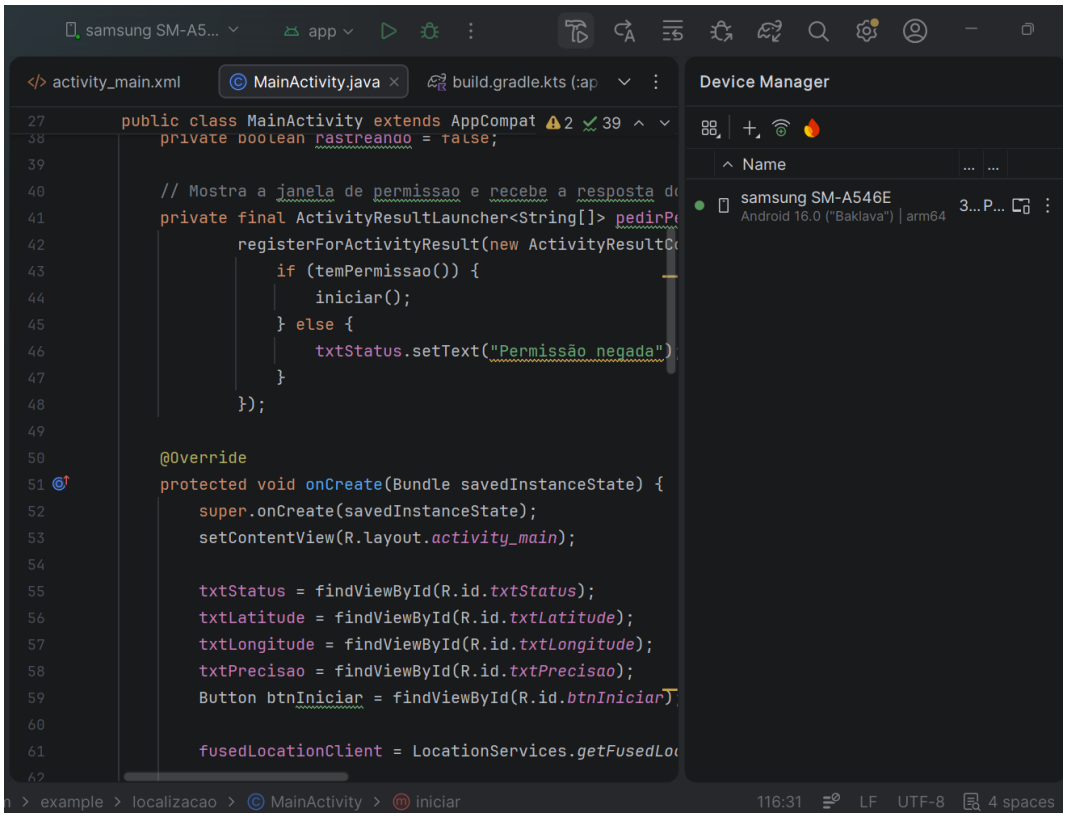
Para testar em um aparelho físico, foram seguidas as etapas abaixo (ANDROID DEVELOPERS).

1. Em **Configurações**, localizar o **Número da versão** e tocar nele sete vezes.
2. Abrir **Opções do desenvolvedor** e ativar a **Depuração USB**.
3. Conectar o celular ao computador pelo cabo USB e aceitar a autorização.
4. Ligar a localização do aparelho.

2.7 Execução do aplicativo

Com o celular selecionado na lista de dispositivos (Figura 7), foi usado o botão **Run** para instalar e abrir o aplicativo.

Figura 7 – Celular selecionado no Android Studio



Fonte: elaborado pela autora (2026).

Ao abrir, o aplicativo solicita o acesso à localização (Figura 8).

Figura 8 – Janela de permissão de localização



Fonte: elaborado pela autora (2026).

3. TESTES E RESULTADOS

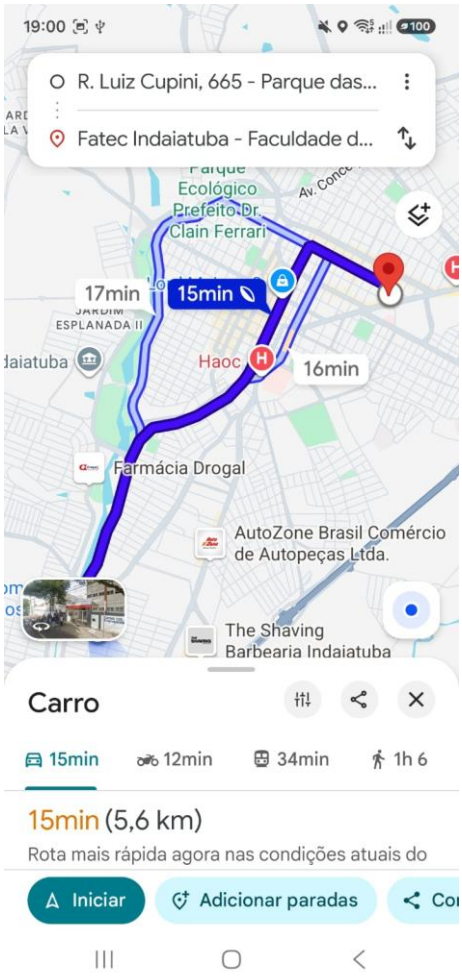
O aplicativo abriu no celular, mostrou as coordenadas com precisão entre 10 e 20 m (Figura 9) e abriu a rota no Google Maps (Figura 10). O Quadro 5 apresenta os testes realizados. O teste de localização aproximada segue os cenários da documentação (ANDROID DEVELOPERS).

Figura 9 – Aplicativo com a localização encontrada e o destino digitado



Fonte: elaborado pela autora (2026).

Figura 10 – Rota aberta no Google Maps



Fonte: elaborado pela autora (2026).

Quadro 5 – Testes do aplicativo

Teste	**Como fazer**	**Resultado esperado**	**Resultado observado**
Permissão precisa	Abrir o aplicativo e escolher Precisa	Coordenadas e precisão na tela	Coordenadas exibidas, com precisão de 10 m
Rota	Digitar "Fatec Indaiatuba" e tocar em Ver rota	Google Maps abre com a rota	Rota de 5,6 km e 15 min de carro
Destino vazio	Tocar em Ver rota sem digitar	Aviso "Digite um destino"	
Permissão aproximada	Desativar a localização precisa nas configurações	Coordenadas com precisão pior, em metros	
Permissão negada	Remover a permissão e negar ao abrir	Aviso "Permissão negada", sem fechar o aplicativo	
Deslocamento	Caminhar alguns metros ao ar livre	Latitude e longitude mudam na tela	

Fonte: elaborado pela autora (2026).

4. CONSIDERAÇÕES FINAIS

O aplicativo mostra que, para usar a localização do celular, é preciso ter a autorização do usuário e pedir ao aparelho que informe sua posição. Depois disso, o aplicativo recebe os dados e os exibe na tela.

O aplicativo também envia essa posição ao Google Maps, que traça o caminho até o destino digitado. Assim, o uso fica simples, pois a localização é buscada assim que o aplicativo abre, e a rota aparece com poucos toques.

5. REFERÊNCIAS

ANDROID DEVELOPERS. **Change location settings**. [S. l.]: Google, [2026]. Disponível em: https://developer.android.com/develop/sensors-and-location/location/change-location-settings. Acesso em: 3 nov. 2026.

ANDROID DEVELOPERS. **Request location access at runtime**. [S. l.]: Google, [2026]. Disponível em: https://developer.android.com/develop/sensors-and-location/location/permissions/runtime. Acesso em: 3 nov. 2026.